

ENTERPRISE SOLUTION MANUAL

DocQuery AI

Knowledge Hub & Q&A RAG Platform

A high-performance web application designed for secure indexing, retrieval, and semantic parsing of corporate internal documentation (PDF, Word, TXT, MD, HTML) leveraging LangChain, ChromaDB, and Google Gemini API.

METADATA CONFIG

Version: 2.1.0 (Stable)

Framework: LangChain v0.1

Vector DB: Chroma DB

DEPLOYMENT SETTINGS

Runtime Server: Streamlit Local Host

Default Port: 8501

URL: <http://localhost:8501>

Developer: Kyzerer & Antigravity AI

Date: July 2026

1. System Architecture & Workflow

DocQuery AI implements a classic Retrieval-Augmented Generation (RAG) pattern. RAG optimizes Large Language Model (LLM) performance by anchoring the models to proprietary enterprise records, preventing hallucinations, and ensuring all generated answers are fully auditable.

1.1 Document Processing & Vector Storage

The workflow begins when a document is uploaded via the Streamlit dashboard. The backend processes the document through the following pipelined stages:

- **Text Extraction:** Determines file extensions and invokes the correct parsing driver (pypdf for PDFs, python-docx for Word files, BeautifulSoup for HTML, or standard readers for TXT/MD files).
- **Text Preprocessing:** Normalizes spacing and line endings to form a clean, continuous block of content.
- **Recursive Chunking:** Segments the cleaned content using characters like paragraph breaks and spaces to ensure high cohesive integrity. Standard size is 1000 characters with 200 characters overlap.
- **Embedding Generation:** Calculates dense vector representations for each text chunk using the Google Generative AI embeddings endpoint.
- **Persistent Storage:** Indexes and stores the generated vectors inside ChromaDB, mapping metadata (like the source filename) back to the chunks.

1.2 Query, Retrieval, and Response Loop

When a user enters a question in the chat interface, the following actions are executed:

- **Semantic Search:** The query is converted into a vector embedding and matched against indexed chunks in ChromaDB using cosine similarity.
- **Context Ingestion:** The top-k (default k=5) matching text blocks are extracted and concatenated to form a context snippet.
- **Factual LLM Assembly:** The system wraps the context and user query inside a strict system prompt, which is sent to the Gemini 2.5 Flash model.
- **Citation Retrieval:** The generated response is displayed alongside the distinct document filenames (citations) that provided the context.

NOTE: This platform does not utilize external cloud databases or third-party web search services. All vector stores and metadata indices are written and read locally on disk, ensuring absolute data privacy and isolation.

2. Core Code Modules & Index

DocQuery AI has been built following clean code conventions. Every system operation is separated into discrete, reusable Python modules:

Module File	System Component Purpose & Implementation Details
app.py	Streamlit UI container; manages session states, sidebars, uploads, deletes, and chat interface.
document_loader.py	Extracts raw text. Uses PyPDFReader, docx.Document, and BeautifulSoup parsing drivers.
text_preprocessor.py	Cleans extracted text, removing double spaces and triple newlines for optimal parsing.
chunker.py	Wraps RecursiveCharacterTextSplitter to chunk text blocks while preserving source metadata.
embeddings.py	Instantiates GoogleGenerativeAIEmbeddings under models/gemini-embedding-001.
vector_store.py	Direct interface for Chroma DB; implements document additions, lists, and chunk-level deletions.
retriever.py	Queries Chroma DB and returns the top k similarity-matched document chunks.
llm_generator.py	Connects to ChatGoogleGenerativeAI (gemini-2.5-flash) and processes context-bounded prompts.

2.1 Local Configuration Template

Authentication keys are loaded automatically from a root .env file. Below is the configuration layout:

```
GEMINI_API_KEY=your_gemini_api_key_here
GOOGLE_API_KEY=your_gemini_api_key_here
```

3. User Manual & Operations Guide

To run and test the application, follow these guidelines to manage files, prompt the model, and verify results.

3.1 Installing & Starting the Application

First, verify that your environment variables are configured in the .env file. Next, execute the installation and launch commands:

```
# Step 1: Install dependencies
pip install -r requirements.txt

# Step 2: Start Streamlit server
python -m streamlit run app.py
```

The server will start and bind locally. You can open and interact with the application by navigating to <http://localhost:8501> in your browser.

3.2 Document Upload & List Verification

1. In the left-hand sidebar, upload any PDF, Word, TXT, MD, or HTML document. You will see a loading spinner during parsing and ingestion.
2. Once ingestion is complete, the document name will appear under the 'Uploaded Knowledge' section in the sidebar.
3. You can verify document registration in the database immediately.

3.3 Chat Q&A and Verification

Submit natural language questions inside the main chat interface. DocQuery AI will query ChromaDB, fetch similar passages, and present the answer. Under the response, a 'Sources Cited' box will display the original filenames.

To test boundary rules (hallucination checks), query the model about a topic not mentioned in your uploaded documents. The LLM should strictly output: 'I cannot find the answer in the provided documents.'

3.4 Document Deletion

To remove a file from the database, click the trash can icon next to its name in the sidebar. The system will delete all vector chunks matching that filename in ChromaDB and trigger a visual refresh.

IMPORTANT: Deleting a document is irreversible. Once deleted, its chunks are purged from ChromaDB and will no longer be retrieved to answer chat queries.